

Does a Small VLA Listen, and Where Does It Break?

Graded robustness of SmolVLA on LIBERO, and a residual RL correction

Anaïs Azouaoui

Sorbonne Université, Paris, France • github.com/ananalab/vla-stress-test

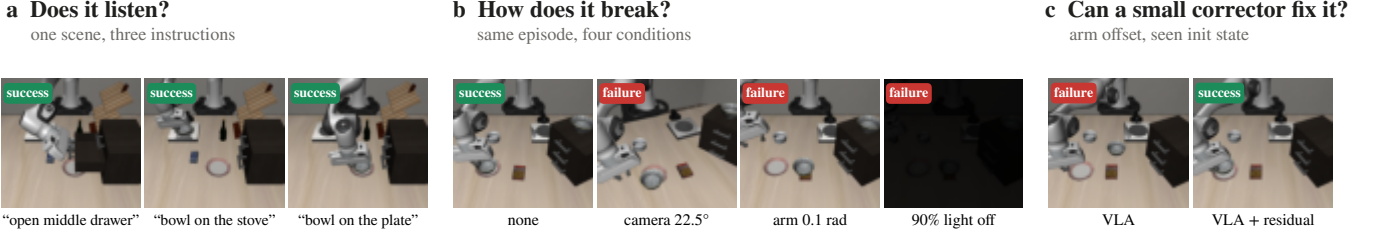


Figure 1: Three questions about a small vision-language-action model. (a) In one LIBERO-Goal scene, the same policy carries out whichever instruction it is given. (b) The same episode fails under a camera rotation, a few degrees of arm offset at the start, or dim light. (c) A residual policy trained by PPO on top of the frozen model can rescue episodes it was trained on. All frames are the model’s own camera input.

Abstract

Vision-language-action (VLA) models are almost always evaluated in the conditions they were trained on. We study SmolVLA [9], a 450M-parameter open VLA, on the LIBERO benchmark [7] through three questions. *Does it listen?* In LIBERO-Goal, where ten tasks share one scene, we swap instructions and evaluate all ten goal predicates at every step: the robot reaches the goal of the instruction it is given in 73% of swapped episodes and the environment’s own goal in 0%, yet hand-written paraphrases of an instruction succeed far less often than its original wording (23% against 80%). *How does it break?* Under five perturbations swept over a continuous intensity, success declines gradually with camera viewpoint, collapses past sharp thresholds for lighting and pixel noise, and is halved by an offset of about 3° per arm joint at the start of the episode. *Can a small corrector help?* A residual policy trained with PPO on top of the frozen model improves success on the initial states it was trained on but not on held-out ones, and mostly learns to finish successful episodes faster. The study comprises 2 439 simulated episodes and about 29 hours of compute on a laptop; all per-episode logs, code and figure scripts are released.

1 Introduction

VLA models map a camera image and a sentence to robot actions with a single network built on a pretrained vision-language model. On simulation benchmarks such as LIBERO their success rates approach saturation, which says little about two questions that matter once the robot leaves the benchmark: is the instruction actually used, and how does performance degrade when the scene departs from the demonstrations? Recent robustness suites [3, 13] address the second question with many perturbation types at a few fixed levels and conclude that VLAs are fragile to viewpoint and initial state and largely ignore language.

We take a narrower and more quantitative look at a single

small model, SmolVLA, chosen because it is open, fits on consumer hardware and is integrated in LeRobot [2]. Our contributions are:

- A **goal-level listening test**. Because all LIBERO-Goal tasks share the same objects, we can evaluate every task’s goal at every step. This separates a robot that ignores the instruction from one that follows it but fails, which success on the environment’s own goal cannot do.
- **Dose-response curves** for five perturbations with a continuous intensity, a breaking point x_{50} estimated in two ways with task-level bootstrap intervals, and per-task curves.
- A **residual RL** experiment targeting the weakness that is visible in proprioception, with an evaluation on held-out initial states that distinguishes learning from generalising.
- A **validated, reproducible pipeline**: a cross-check against the official evaluation, episode-level determinism checks, tests that each perturbation changes the policy input, and a paired design in which every condition is evaluated on the same episodes.

2 Related work

Vision-language-action models. OpenVLA, π_0 and related models fine-tune large vision-language backbones to output actions; SmolVLA [9] reduces the backbone to a truncated 500M SmolVLM2 and adds a flow-matching [6] action expert that predicts chunks of 50 actions, reaching performance comparable to much larger models on LIBERO.

Robustness of VLAs. LIBERO-Plus [3] perturbs LIBERO along seven axes (camera, robot initial state, language, lighting, background, sensor noise, layout) and reports that camera and initial-state changes are the most damaging while language variations matter little, which the authors interpret as models ignoring language. LIBERO-PRO [13] shows that success can collapse under object, position and instruction changes and attributes high scores to memorisation. We reuse their axes but sweep intensity continuously, and replace success on the

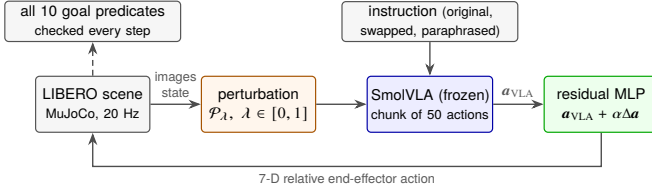


Figure 2: Evaluation loop. Perturbations act on the simulator after each reset (camera, lights, arm pose) or on the images given to the policy (noise, masking); the instruction is replaced for the language tests. The residual corrector is only used in Section 6; elsewhere $a = a_{\text{VLA}}$.

environment’s goal by an evaluation of all goals of the shared scene.

Residual policy learning. Residual RL learns an additive correction on top of a fixed controller [5, 10]. ResiP [1] and Policy Decorator [12] refine imitation-learned diffusion or transformer policies this way, with a correction bounded in magnitude and initialised to zero. We apply the same recipe to a VLA, targeting one specific robustness failure.

3 Experimental setup

3.1 Model, simulator and evaluation loop

We use the Hub checkpoint `lerobot/smolvla_libero` (revision 31d453f), fine-tuned on the LIBERO demonstrations with LeRobot. The policy receives the third-person (*agentview*) and wrist images at 360×360 , an 8-D state (end-effector position, axis-angle orientation, gripper opening) and the instruction, and outputs a chunk of 50 relative end-effector actions sampled with 10 flow-matching steps; the whole chunk is executed before the next query, as in the checkpoint’s configuration. Observations go through the pre- and post-processors saved with the checkpoint (normalisation, tokenisation, camera renaming), so the model sees exactly what it would see under `lerobot-eval`. Episodes end on success or after 280 (Spatial) or 300 (Goal) steps.

Everything ran on an Apple M1 laptop with 8 GB of memory: LIBERO runs natively on macOS with offscreen CGL rendering once an unused Linux-only dependency is skipped, and the policy runs on the Metal backend with the checkpoint’s bf16 weights (Table 6). We use LIBERO-Spatial for perturbations (ten tasks with distinct layouts) and LIBERO-Goal for language (ten tasks in one scene).

3.2 Protocol

Episode e of a task always starts from LIBERO’s e -th fixed initial state with random seed e , which also fixes the flow-matching noise, whatever the condition. Two conditions therefore differ only by the perturbation, and outcomes can be compared episode by episode. Every episode uses a hard reset (the MuJoCo model is rebuilt), and each row of the result files records configuration, seed, commit, library versions, checkpoint revision and device. Baseline and language experiments use 10 and 3–5 episodes per task; dose-response experiments use 5 episodes per task and level (50 per level).

Table 1: Unperturbed success. Wilson 95% intervals in brackets. The reported numbers may refer to a different checkpoint than the one on the Hub.

Suite	This work (10 tasks $\times$ 10)	Reported [9]
LIBERO-Spatial	59/100 (59%) [49, 68]	90%
LIBERO-Goal	84/100 (84%) [76, 90]	92%
Cross-check on Spatial tasks 1 and 3: <code>lerobot-eval</code> 14/20, this loop 12/20.		

3.3 Statistics

Success rates are reported with Wilson 95% intervals [11], which stay inside $[0, 1]$ and behave well for small n and extreme rates. Paired conditions are compared with an exact McNemar test on discordant episodes. For a perturbation of intensity x (in physical units) we fit

$$p(x) = \frac{s_0}{1 + \exp((x - x_{50})/w)} \quad (1)$$

by binomial maximum likelihood with s_0 fixed to the unperturbed success rate on the same episodes, so that x_{50} is the intensity at which success is halved. Because a logistic cannot represent every shape (a plateau followed by a cliff and a long tail, for instance), we also report a model-free x_{50} : pooled success per level is made non-increasing by isotonic regression and linearly interpolated at $s_0/2$. Intervals on both estimates come from a bootstrap over *tasks* (300 resamples of whole tasks), since task difficulty varies far more than outcomes within a task.

3.4 Validating the pipeline

Official evaluation. On two LIBERO-Spatial tasks the official `lerobot-eval` gives 14/20 (70%) and our loop 12/20 (60%) (Fisher $p = 0.74$).

Determinism. Re-running recorded episodes reproduces their outcome *and* their exact length, on the Metal backend and after rebuilding the Python environment from scratch.

Perturbations reach the policy. Unit tests check that each perturbation changes the policy’s input image, monotonically with intensity, and is the identity at intensity 0. The first version of the camera and lighting perturbations silently did nothing: camera poses are only recomputed by MuJoCo’s forward pass, and robosuite caches observations between simulation steps. Without the image check the dose-response curves would have been flat for the wrong reason. For language, we decode the token sequence actually passed to the model.

Baselines. Table 1 gives 59/100 (59%, CI 49–68) on LIBERO-Spatial and 84/100 (84%, CI 76–90) on LIBERO-Goal, below the 90% and 92% reported in [9]. The cross-check above rules out our loop as the cause; the Hub checkpoint was trained separately from the paper’s runs. All conclusions below are relative to the measured baseline, and tasks vary widely (Table 4).

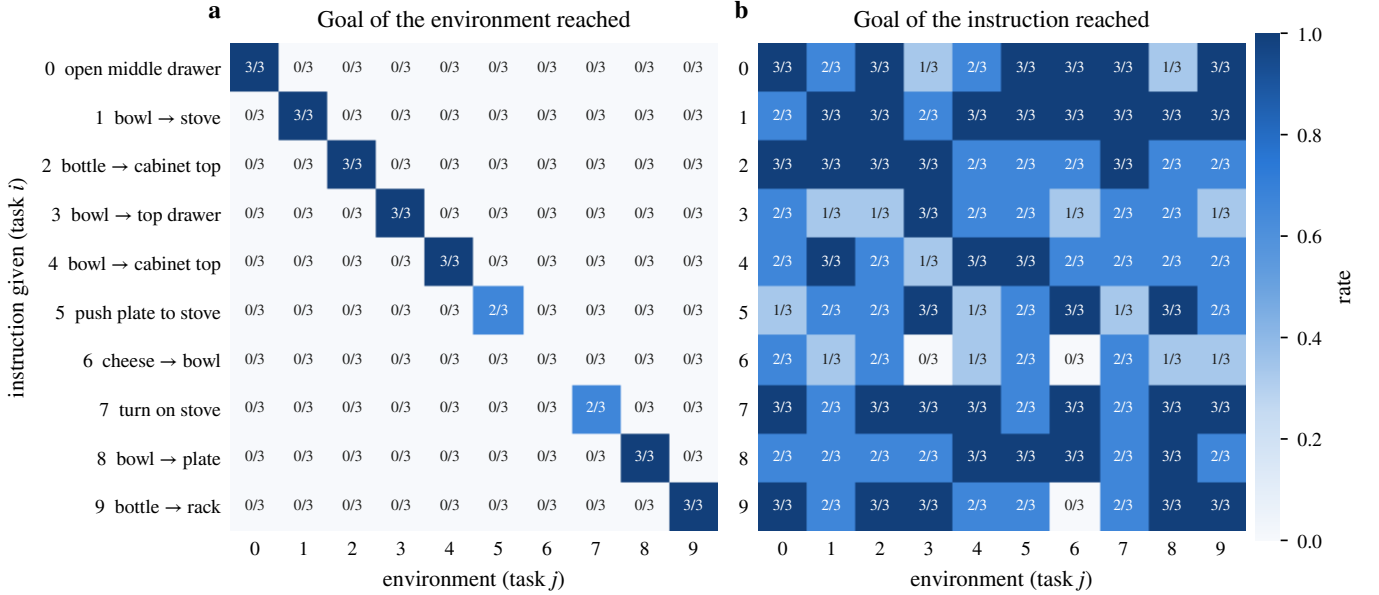


Figure 3: Listening test (3 episodes per cell, 300 episodes). The environment of task j is built and the instruction of task i is given. (a) Goal j reached, as scored by the environment. (b) Goal of the *given* instruction i reached at any step. Off the diagonal, the robot never completes the environment’s task but completes the instructed one in 197/270 (73%, CI 67–78) of episodes.

4 Does the model listen?

4.1 Swapping instructions in a shared scene

All ten LIBERO-Goal tasks are defined on the same objects (a bowl, a plate, a bottle, a cream cheese box, a stove, a cabinet with drawers, a rack) and differ only by their instruction, goal predicate and set of initial states. We build the environment of task j , which scores goal j , give the instruction of task i , and in addition evaluate the ten goal predicates at every step. M_{ij} is either the rate at which goal j is reached (what the environment reports) or the rate at which the instructed goal i is reached.

Read alone, Figure 3a is ambiguous: success only on the diagonal is what a robot that ignores language would also produce if its default behaviour did not happen to satisfy goal j . Figure 3b resolves it. Told to do task i in the scene of task j , the robot reaches goal i in 73% of the 270 swapped episodes, close to the rate with matched instructions (25/30 (83%, CI 66–93)), never reaches the environment’s goal (0/270), and completes a third, unrelated goal in only 2/270 episodes. The initial states of the ten tasks differ slightly, so the behaviour is not keyed on the starting configuration either. Instructions involving cluttered targets are followed less reliably (task 3, bowl into the top drawer; task 6, cream cheese into the bowl). Keyframes of one scene under four instructions are shown in Figure 4.

4.2 ... but to the exact words

We then keep the goal and change the words (Figure 6). Three hand-written paraphrases per task, from close to fully reworded, succeed in 35/150 (23%, CI 17–31) of episodes against 40/50 (80%, CI 67–89) with the original wording on the same episodes, with a clear gradient: 20/50 close, 11/50 reworded, 4/50 distant. Small changes matter: “*turn the stove on*” works, “*switch on*

the stove” never does. An empty instruction and an unrelated one (“*sing a song*”) give 0/50 and 0/50, and in none of the 215 failed episodes is any of the ten goals reached: without an instruction it recognises, the robot does not fall back on a default task.

Interpretation. The instruction is what selects the behaviour, but it acts more like a key into a small set of learned behaviours than like language that is understood. For this model and suite we find sensitivity to language on both counts, naming another goal changes the behaviour and rewording the same goal breaks it, which differs from the conclusion of LIBERO-Plus. Models, suites and perturbation types all differ, and we did not reproduce their setting; a swap test in a shared scene is simply the most direct check that the instruction is used.

5 How does it break?

5.1 Graded perturbations

Each perturbation maps an intensity $\lambda \in [0, 1]$ linearly to a physical magnitude $m = \lambda m_{\max}$ (Figure 8).

- **Camera orbit** ($m_{\max} = 60^\circ$). The agentview camera’s optical axis is intersected with the table plane at $\mathbf{c}$; position and orientation are rotated by $R_z(\pm m)$ about the vertical axis through $\mathbf{c}$, so the camera keeps looking at the same point. The sign alternates with the episode. The wrist camera is untouched.
- **Arm initial pose** ($m_{\max} = 0.2$ rad). After reset, each of the seven joints is shifted by $s_k m$ with signs $s_k \in \{\pm 1\}$ drawn once per episode, followed by five settling steps. Objects do not move.
- **Lighting** ($m_{\max} = 0.9$). Diffuse, ambient and specular components of both scene lights and of the headlight are multiplied by $1 - m$.

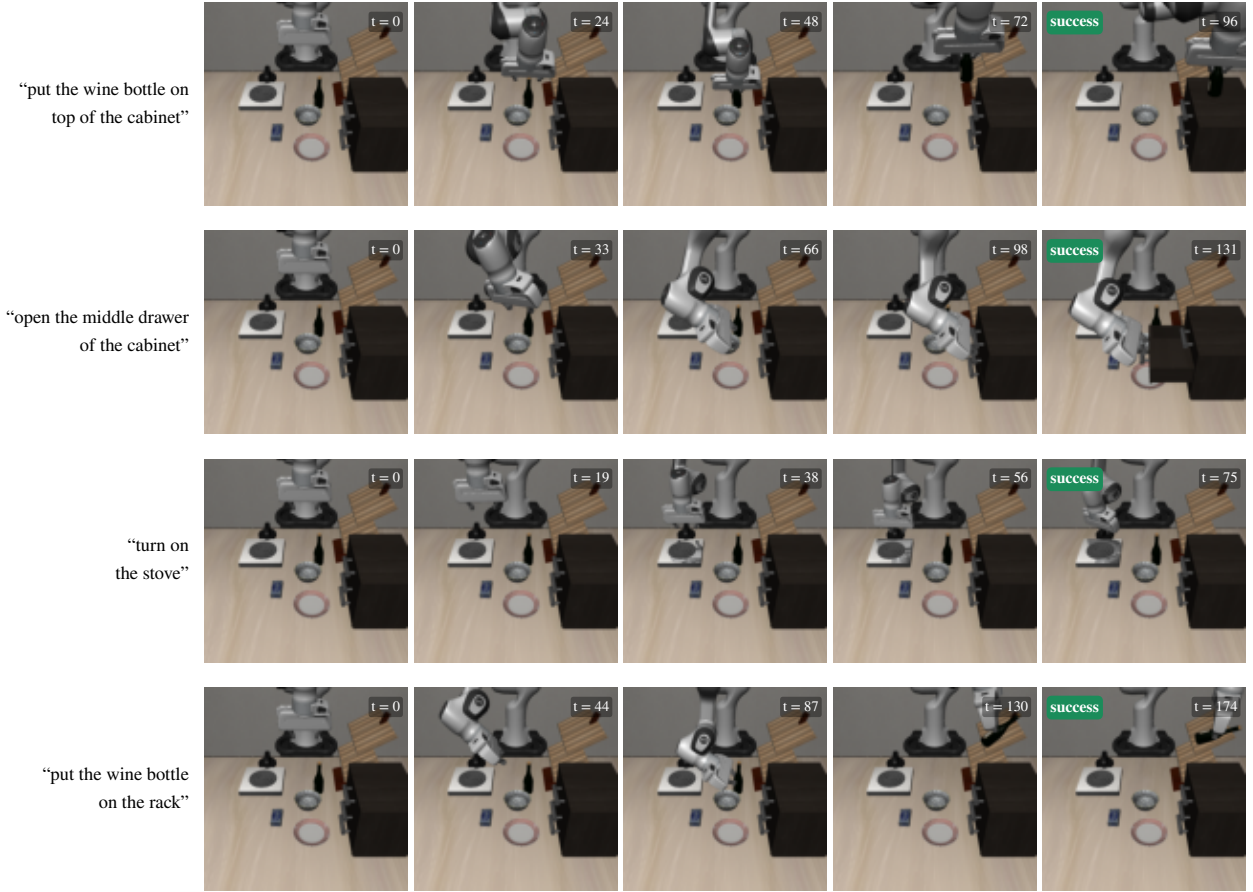


Figure 4: One scene, four instructions. LIBERO-Goal environment of task 2 (whose own instruction is the first row), initial state 1. Each row is a full episode, frames evenly spaced in time.

Table 2: Breaking points, with 95% task-bootstrap intervals. Levels include the unperturbed one.

Perturbation (unit)	Logistic x_{50}	Model-free x_{50}	Shape
Camera orbit (deg)	39.7 [24.7, 61.2]	39.4 [15, >60]	gradual
Arm initial pose (rad)	0.0782 [0.0504, 0.109]	0.0477 [0.0305, 0.0821]	tolerance, then cliff
Lighting (fraction off)	0.761 [0.724, 0.809]	0.744 [0.716, 0.782]	threshold
Pixel noise (noise std)	42.3 [35.8, 45.8]	41.7 [34.9, 47.2]	threshold

- **Pixel noise** ($m_{\max} = 60$). Gaussian noise of standard deviation m (0–255 scale) on both camera images, with a fixed seed per episode.
- **Camera masking**: one of the two images is replaced by black.

Simulator-level changes are re-applied after every reset, since a hard reset rebuilds the model; with soft resets (used for RL training) the original values are cached so that changes do not compound. At the largest intensity all task-relevant objects remain visible, except at 90% light removal where the scene is very dark but still readable.

5.2 Results

The curves have different *shapes*, not just different scales (Figure 5, top; Table 2):

- **Camera orbit: gradual.** Success declines steadily with no

plateau and no cliff, is halved around 40° and remains above zero at 60° . The two orbit directions are not equally harmful at intermediate angles, but direction is confounded with the initial state (it alternates with the episode index), so we do not interpret this.

- **Arm initial pose: narrow tolerance, then a cliff.** Offsets up to 0.02 rad per joint ($\approx 1^\circ$) leave success unchanged; it is halved by 0.048 rad [0.031, 0.082] (model-free; the logistic, which cannot follow both the cliff and the long tail, gives 0.078), although the objects have not moved and both cameras still see them.
- **Lighting: threshold.** No measurable effect up to 67.5% of the light removed; success is then halved at 0.74 and falls below a quarter of the baseline at 81%.
- **Pixel noise: threshold.** No effect at a standard deviation of 15, a small one at 30, halved at 42 [35, 47] and near zero at 60, a level at which a human still sees every object (Figure 8).

Per-task heterogeneity. The bottom row of Figure 5 shows what the pooled curves average over. The photometric thresholds are shared: for lighting all tasks fail between the same two levels. The viewpoint curve is the opposite: some tasks (0 and 2) are unaffected even at 60° while others (7 and 9) fail from the first levels, so the pooled “gradual” decline is in part a mixture of task-specific cliffs. The arm curve mixes both, with

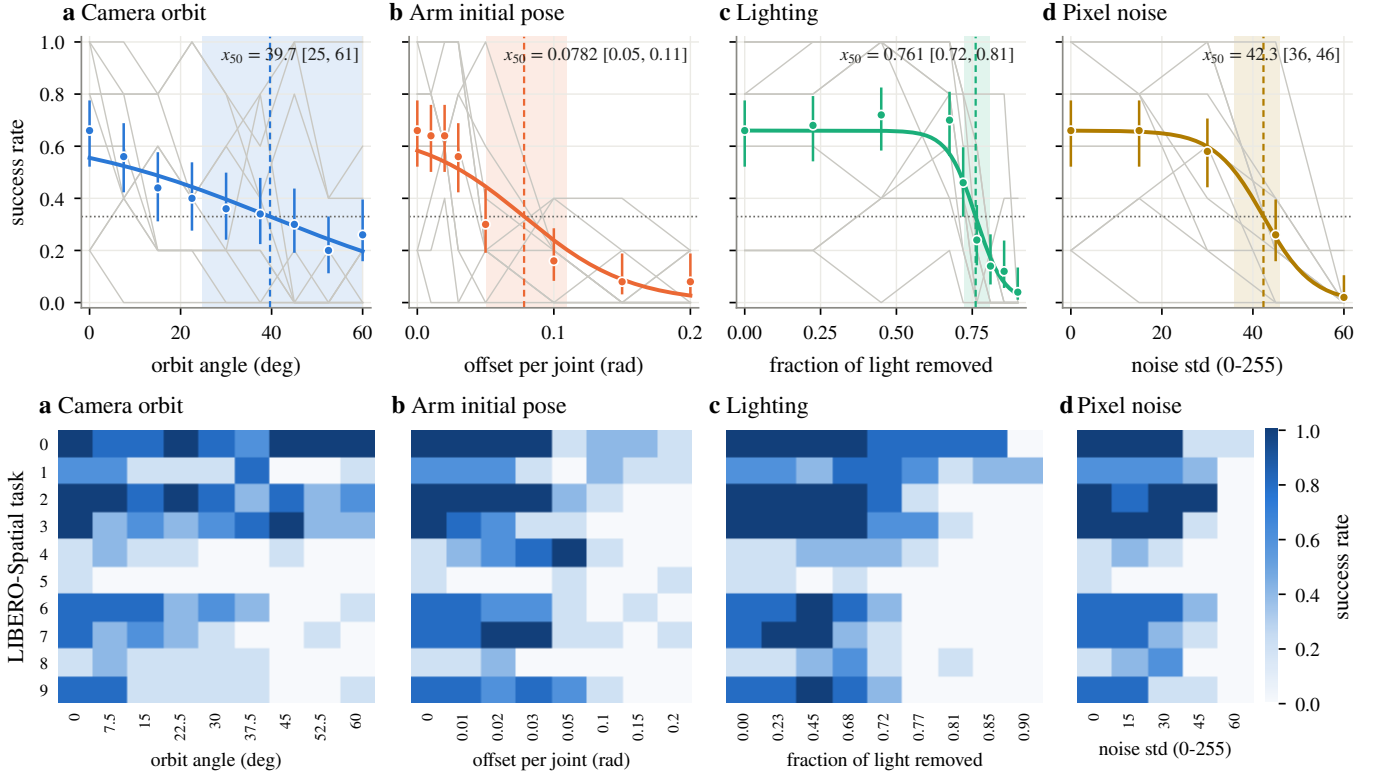


Figure 5: Success versus perturbation intensity on LIBERO-Spatial (10 tasks $\times$ 5 episodes per level; the unperturbed point uses the same episodes). *Top*: pooled success with Wilson 95% intervals, individual tasks in grey, logistic fit (Equation (1)) with half the baseline dotted, and logistic x_{50} with its 95% task-bootstrap interval (dashed line and band). *Bottom*: success per task and level (5 episodes per cell), same levels as the top row.

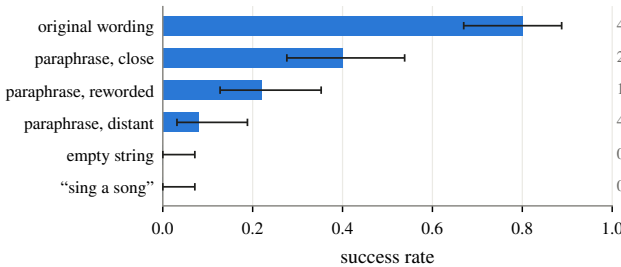


Figure 6: Instruction variants on LIBERO-Goal (environment and instruction of the same task, 5 episodes per task and variant). Paraphrases were written by hand (Table 7).

most tasks holding up to 0.03 rad and dropping together.

Both cameras are needed. Blacking out one stream entirely drops success from 33/50 to 5/50 without the third-person view and 6/50 without the wrist view. The policy cannot do without the third-person image, yet tolerates rotating it by 20–30° with a moderate loss, which suggests it relies on the coarse layout of that view rather than on precise image coordinates.

What failures look like. Figure 7 follows one episode under each perturbation. The failures are not random motions: under the camera rotation the policy carries out the whole pick-and-place but puts the bowl down next to the plate, as if its estimate of the plate position were shifted; with the arm offset it never

grasps the bowl and the gripper ends up out of view.

6 Can a small corrector fix it?

6.1 Residual RL formulation

We target the arm-pose weakness, which is the one visible in proprioception: a corrector without images can in principle detect it. The task is LIBERO-Spatial task 2 at 0.05 rad per joint, where the VLA alone succeeds in about half of the episodes. At step t the corrector observes $\mathbf{o}_t = (\mathbf{p}_t, \mathbf{q}_t, \mathbf{g}_t, \mathbf{a}_t^{\text{VLA}}, t/T) \in \mathbb{R}^{17}$ (end-effector position and quaternion, gripper opening, the VLA’s action for this step, progress) and outputs $\Delta \mathbf{a}_t \in [-1, 1]^7$; the executed action is

$$\mathbf{a}_t = \text{clip}(\mathbf{a}_t^{\text{VLA}} + \alpha \Delta \mathbf{a}_t, -1, 1), \quad \alpha = 0.2. \quad (2)$$

The reward is 1 on success and 0 otherwise; episodes end on success or after $T = 280$ steps. Actor and critic are separate MLPs (Table 5); the actor’s last layer is initialised to zero, so before training the corrected policy is the VLA, which we verified on full episodes (identical outcomes and episode lengths). We train with PPO [8] in a single-file implementation in the style of CleanRL [4], four simulators in one process sharing batched VLA queries (about 10 steps/s), soft resets during training, and 100k environment steps per run (about 400 episodes). Initial states 0–39 are used for training; 40–49 are held out. Evaluation uses the deterministic mean action and hard resets, like every other number in this report.

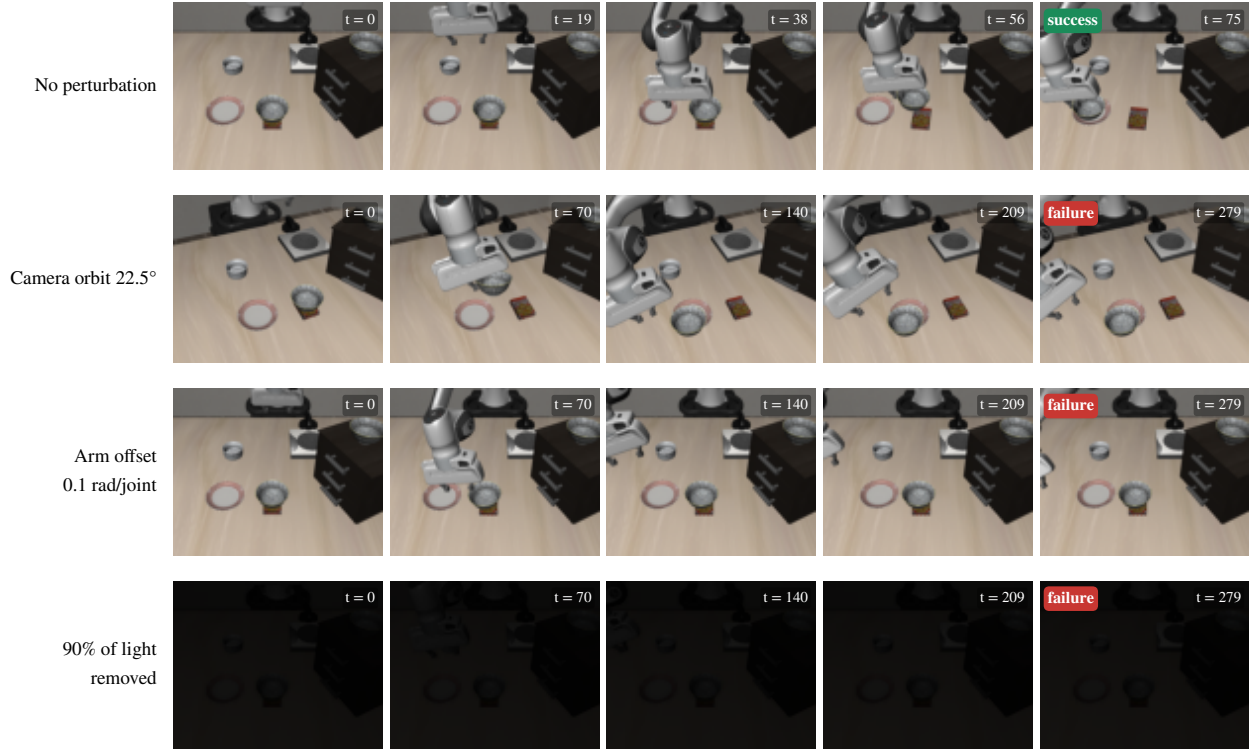


Figure 7: How failures unfold (LIBERO-Spatial task 3, initial state 2, same seed in every row). Under the camera orbit the bowl is picked up but ends next to the plate; with the arm offset the bowl is never grasped and the gripper drifts out of view; in the dark the bowl is never grasped either.

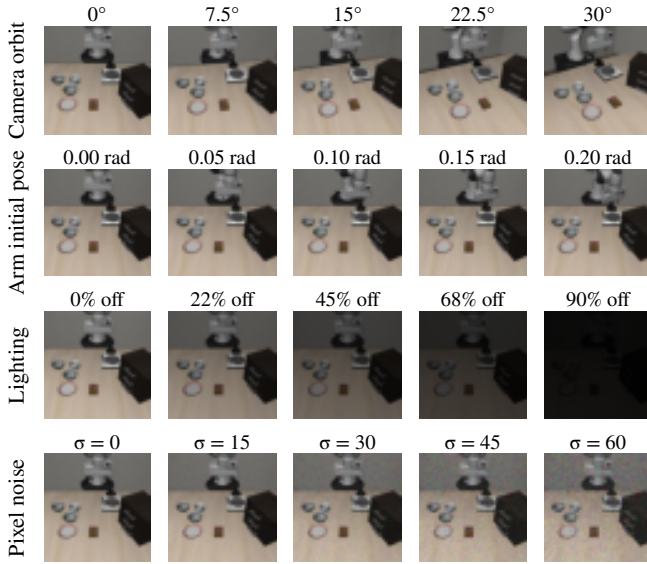


Figure 8: Policy input at intensities 0, 0.25, 0.5, 0.75 and 1 (LIBERO-Spatial task 0, initial state 0; camera shown up to 30°).

6.2 Results

Training success rises modestly (from 0.60, 0.51, 0.54 to 0.70, 0.66, 0.71 over the first and last ten updates of each of the three seeds; Figure 9a), and two of the three runs dip around 70k steps before recovering. On initial states seen in training, the deterministic corrector succeeds in 14/20, 13/20, 13/20 episodes against 10/20 for the VLA alone, and every discordant

Table 3: Residual RL evaluation (successes / episodes, offset in rad per joint). “Re-plan / 10” executes 10 instead of 50 actions per chunk, without any learning.

Initial states	Offset (rad)	VLA	+ res. s0	+ res. s1	+ res. s2	Re-plan / 10
Seen (0–19)	0.05	10/20	14/20	13/20	13/20	–
Held-out (40–49)	0.05	5/10	4/10	5/10	4/10	4/10
Held-out (40–49)	0	8/10	8/10	10/10	9/10	10/10
Held-out (40–49)	0.10	1/10	1/10	1/10	1/10	3/10
Seen: steps per success		110	91	90	88	
Seen: McNemar p vs VLA			0.12	0.38	0.38	
Held-out: steps per success		95	89	92	88	
Held-out: McNemar p vs VLA			1.00	1.00	1.00	

episode favours the corrector (Table 3; McNemar $p = 0.12, 0.38, 0.38$, not significant at this sample size). On held-out initial states the gain disappears: 4/10, 5/10, 4/10 against 5/10 at the training level and 1/10, 1/10, 1/10 against 1/10 at 0.10 rad. The corrector does not hurt the unperturbed task (8/10, 10/10, 9/10 against 8/10).

What is learned. What transfers is speed: successful episodes are shorter with the corrector, from 110 to 88–91 steps on seen states and from 95 to 88–92 on held-out ones. This is what the objective rewards most directly: with discounting a faster success is worth more, and an episode that already works can be sped up in every rollout, whereas recovering a failure is a rare event in 400 episodes. The recoveries that do occur are tied to the configurations seen in training; with 40 of them, a proprioceptive corrector has little to interpolate from.

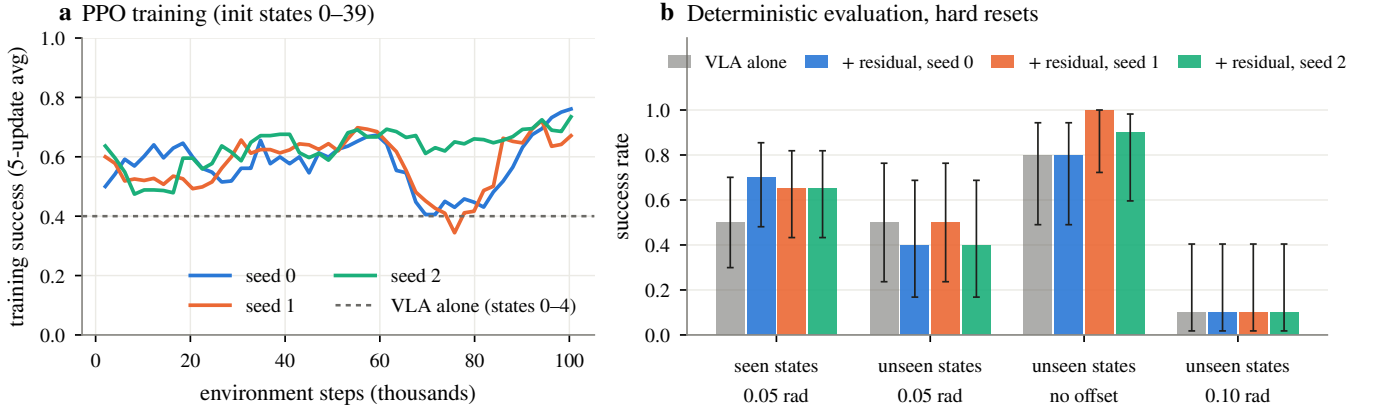


Figure 9: Residual RL on the arm-offset weakness. (a) Success of the stochastic policy during PPO, moving average over 5 updates. (b) Deterministic evaluation with hard resets on initial states seen in training (0–19) and held out (40–49). Wilson 95% intervals.

A training-free alternative. Executing 10 instead of 50 actions per chunk, so that the VLA re-plans five times more often, gives 10/10, 4/10 and 3/10 on the held-out episodes (no offset, 0.05 and 0.10 rad), not distinguishable from the VLA alone with 10 episodes, at 2.4× the cost.

7 Limitations and threats to validity

- **Scope.** One model, one checkpoint, two LIBERO suites, simulation only. The measured baselines are below published numbers; we report changes relative to them.
- **Sample size.** 3–10 episodes per cell. Intervals are wide and many per-task differences are within noise; we report intervals rather than significance stars and do not correct for multiple comparisons.
- **Shared episodes.** Using the same initial states and seeds across conditions removes noise from comparisons but correlates them: a hard initial state is hard in every condition.
- **Numerics.** Inference ran on Apple’s Metal backend in bf16; results on CUDA may differ slightly. A GPU runner with the same configurations is provided but was not used for reported numbers.
- **Paraphrases** were written by one person and are not a sample of how users phrase tasks.
- **Camera direction** is confounded with the initial state; we only interpret magnitude.
- **Residual RL** used one task, one perturbation level, about 400 training episodes per run and soft resets during training, so its negative result says more about this budget than about residual RL.

8 Conclusion

On LIBERO, SmolVLA is neither deaf to language nor robust to it: the instruction decides which task is attempted, but only in wordings close to those seen in training. Its failures under distribution shift have distinct shapes, gradual for viewpoint, abrupt for lighting, noise and arm configuration, which a single perturbation level per axis would hide, and pooled curves partly average over task-specific cliffs. A residual corrector trained on a few hundred episodes does not repair the arm-pose weakness

on unseen configurations; making it generalise likely requires many more initial states or an input that encodes the scene, not only proprioception. Natural next steps are a search over combinations of perturbations instead of one axis at a time, and the same protocol on larger VLAs.

Reproducibility. Every number in this report is generated from the per-episode CSV files by scripts in the repository, which also contains the configurations, the trained correctors and the rollout videos.

References

- [1] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal. From imitation to refinement – residual RL for precise assembly. *arXiv preprint arXiv:2407.16677*, 2024.
- [2] R. Cadene et al. LeRobot: State-of-the-art machine learning for real-world robotics in pytorch. <https://github.com/huggingface/lerobot>, 2024.
- [3] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu. LIBERO-Plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [4] S. Huang, R. F. J. Dossa, C. Ye, J. Braga, D. Chakraborty, K. Mehta, and J. G. Araújo. CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274):1–18, 2022.
- [5] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine. Residual reinforcement learning for robot control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2019.
- [6] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [7] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023.
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [9] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma,

- A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [10] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling. Residual policy learning. *arXiv preprint arXiv:1812.06298*, 2018.
- [11] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [12] X. Yuan, T. Mu, S. Tao, Y. Fang, M. Zhang, and H. Su. Policy decorator: Model-agnostic online refinement for large policy model. *arXiv preprint arXiv:2412.13630*, 2024.
- [13] X. Zhou, Y. Xu, G. Tie, Y. Chen, G. Zhang, D. Chu, P. Zhou, and L. Sun. LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.

A Additional tables

Table 4: Unperturbed successes per task (out of 10).

Task	0	1	2	3	4	5	6	7	8	9	All
Spatial	10	6	7	6	4	1	8	8	2	7	59/100
Goal	10	10	9	8	8	8	4	9	8	10	84/100

Table 5: PPO configuration, read from the saved checkpoints.

Task / perturbation	Spatial task 2, arm offset 0.05 rad/joint
Correction scale α	0.2
Policy / value network	MLP, 2×128 tanh, separate
Initial std of Δa	e^{-1}
Parallel environments	4
Steps per env per rollout	512
Total environment steps	100 000
Epochs / minibatch	10 / 256
Learning rate (Adam)	0.0003
γ / GAE λ	0.995 / 0.95
Clip / value coef. / entropy coef.	0.2 / 0.5 / 0.0
Max grad norm	0.5
VLA actions per chunk executed	50
Training init states	0–39

Table 6: Compute. Wall-clock time on an Apple M1 laptop (8 GB), one process at a time.

Experiment	Episodes	s / episode	Hours
Baselines	200	29	1.6
Listening matrix	300	26	2.2
Instruction variants	250	37	2.6
Dose-response (4 families)	1350	27	10.2
Camera masking	100	31	0.9
Residual RL evaluation	235	24	1.6
Validation re-runs	4	12	0.0
PPO training (3 runs)	–	–	9.8
Total	2439		28.8

Table 7: Paraphrases used in Figure 6, with successes out of 5 (original wording: episodes 0–4 of the baseline).

Task	Original		Close		Reworded		Distant	
0	open the middle drawer of the cabinet	5/5	open the drawer in the middle of the cabinet	0/5	pull the cabinet’s middle drawer open	0/5	slide out the middle drawer	1/5
1	put the bowl on the stove	5/5	place the bowl on the stove	4/5	move the bowl onto the stove	5/5	set the bowl down on the cooktop	0/5
2	put the wine bottle on top of the cabinet	4/5	place the wine bottle on top of the cabinet	4/5	move the bottle of wine onto the cabinet	0/5	set the bottle on the cabinet top	0/5
3	open the top drawer and put the bowl inside	4/5	open the top drawer and place the bowl inside	2/5	pull out the upper drawer, then put the bowl in it	0/5	put the bowl in the top drawer after opening it	0/5
4	put the bowl on top of the cabinet	4/5	place the bowl on top of the cabinet	3/5	move the bowl onto the cabinet	0/5	set the bowl down on the cabinet top	2/5
5	push the plate to the front of the stove	4/5	push the plate to the front of the cooktop	0/5	slide the plate in front of the stove	0/5	move the plate forward until it is in front of the stove	0/5
6	put the cream cheese in the bowl	1/5	place the cream cheese in the bowl	1/5	move the cream cheese into the bowl	0/5	drop the cheese box into the bowl	0/5
7	turn on the stove	4/5	switch on the stove	0/5	turn the stove on	4/5	start the cooktop	0/5
8	put the bowl on the plate	4/5	place the bowl on the plate	3/5	move the bowl onto the plate	2/5	set the bowl down on the dish	0/5
9	put the wine bottle on the rack	5/5	place the wine bottle on the rack	3/5	move the bottle of wine onto the rack	0/5	set the bottle in the wine rack	1/5